**1) (i) Write code for understanding data types in Python.**
**(ii) Develop a program to find the factorial to understand control structures and iteration.**
**(iii) Develop programs to learn Fibonacci series using branching in Python.**

**# Program: Understanding Data Types in Python**

```
# Integer type
a = 10
print("Integer value:", a, "| Type:", type(a))

# Float type
b = 10.5
print("Float value:", b, "| Type:", type(b))

# String type
c = "Hello Python"
print("String value:", c, "| Type:", type(c))

# Boolean type
d = True
print("Boolean value:", d, "| Type:", type(d))

# List type (collection of multiple data)
e = [1, 2, 3, 4, 5]
print("List value:", e, "| Type:", type(e))

# Tuple type (ordered, immutable collection)
f = (10, 20, 30)
print("Tuple value:", f, "| Type:", type(f))

# Set type (unordered collection of unique elements)
g = {1, 2, 3, 4}
print("Set value:", g, "| Type:", type(g))

# Dictionary type (key-value pairs)
h = {"name": "Urvisha", "course": "BCA"}
print("Dictionary value:", h, "| Type:", type(h))
```

**Output:**
Integer value: 10 | Type: <class 'int'>
Float value: 10.5 | Type: <class 'float'>

```
String value: Hello Python | Type: <class 'str'>
Boolean value: True | Type: <class 'bool'>
List value: [1, 2, 3, 4, 5] | Type: <class 'list'>
Tuple value: (10, 20, 30) | Type: <class 'tuple'>
Set value: {1, 2, 3, 4} | Type: <class 'set'>
Dictionary value: {'name': 'Urvisha', 'course': 'BCA'} | Type: <class 'dict'>
```

**(ii) Program to Find Factorial Using Control Structure and Iteration**
**# Program: Factorial using control structure and iteration**

```
# Input number from user
num = int(input("Enter a number to find factorial: "))

# Initialize result variable
fact = 1

# Use 'for' loop for iteration
for i in range(1, num + 1):
    fact = fact * i  # Multiply each number till n

# Display factorial result
print("Factorial of", num, "is:", fact)
```

Output:
Enter a number to find factorial: 5
Factorial of 5 is: 120
Explanation:

Loop runs from 1 to num.

Each iteration multiplies fact with i.

Final value of fact gives the factorial.

**(iii) Program for Fibonacci Series Using Branching (if-else)**
**# Program: Fibonacci series using branching**

```
# Input number of terms
n = int(input("Enter number of terms: "))

# First two terms of Fibonacci series
a, b = 0, 1
```

```
count = 0

# Check if the number of terms is valid
if n <= 0:
    print("Please enter a positive integer")
elif n == 1:
    print("Fibonacci sequence up to", n, "term:")
    print(a)
else:
    print("Fibonacci sequence:")
    while count < n:
        print(a, end=" ")  # Print current term
        c = a + b          # Next term = sum of previous two
        a = b              # Shift a to b
        b = c              # Shift b to new term
        count += 1         # Increment counter
```

Output:
Enter number of terms: 7
Fibonacci sequence:
0 1 1 2 3 5 8

**Explanation:**

If n <= 0, invalid input.

If n == 1, only first term printed.

Else, while loop prints the series by summing the last two terms repeatedly.

**2) (i) Develop programs for printing 1–100 without using a loop.**
**(ii) Develop programs for accessing the values of global variables in different functions.**

**(i) Program to Print 1–100 Without Using a Loop**

We'll use recursion (a function calling itself) to print numbers from 1 to 100 without using any loop (for or while).

**# Program: Print numbers from 1 to 100 without using a loop**

```
def print_numbers(n):
    # Base condition: Stop when n > 100
    if n > 100:
        return
    else:
        print(n)            # Print current number
        print_numbers(n + 1)  # Recursive call to print next number

# Function call starting from 1
print_numbers(1)
```

**Output:**

```
1
2
3
4
...
99
100
```

**Explanation:**

print_numbers() is called recursively.

Each time it prints the current value of n and calls itself with n + 1.

When n exceeds 100, the recursion stops (base condition).

This technique uses function recursion instead of loops to achieve iteration.

**(ii) Program for Accessing Global Variables in Different Functions**

This demonstrates how to access and modify global variables from within multiple functions.

```
# Program: Accessing global variables in different functions

# Declare a global variable
count = 10

def display():
    # Access global variable (read only)
    print("Inside display() function, count =", count)
```

```
def modify():
    global count  # Use 'global' keyword to modify global variable
    count = count + 5
    print("Inside modify() function, count updated to =", count)

# Main program
print("Global variable initially:", count)
display()    # Access global variable
modify()     # Modify global variable
print("Global variable after modification:", count)
```

**Output:**
Global variable initially: 10
Inside display() function, count = 10
Inside modify() function, count updated to = 15
Global variable after modification: 15

**Explanation:**

The variable count is global, declared outside all functions.

Inside display(), we just read the global variable.

Inside modify(), we use the global keyword to modify it.

The change reflects globally across the program.

**3) (i) Develop programs to learn Random modules.**
**(ii) Develop a program to print the current date & time and use some math functions.**

**(i) Program to Learn random Module**

The random module in Python is used to generate random numbers and make random selections — useful in games, simulations, and data sampling.

```
# Program: Understanding the 'random' module in Python

# Import the random module
import random
```

```
# 1 Generate a random integer between 1 and 10
num1 = random.randint(1, 10)
print("Random integer between 1 and 10:", num1)

# 2 Generate a random floating-point number between 0 and 1
num2 = random.random()
print("Random float between 0 and 1:", num2)

# 3 Generate a random number within a specific range with a step
num3 = random.randrange(5, 50, 5)
print("Random number between 5 and 50 (step 5):", num3)

# 4 Choose a random element from a list
fruits = ['apple', 'banana', 'cherry', 'mango', 'orange']
choice = random.choice(fruits)
print("Randomly selected fruit:", choice)

# 5 Shuffle the list randomly
random.shuffle(fruits)
print("Shuffled fruit list:", fruits)
```

Output:
Random integer between 1 and 10: 7
Random float between 0 and 1: 0.3842762921
Random number between 5 and 50 (step 5): 25
Randomly selected fruit: mango
Shuffled fruit list: ['banana', 'apple', 'orange', 'cherry', 'mango']

**Explanation:**

random.randint(a, b) → Random integer between a and b.

random.random() → Random float between 0 and 1.

random.randrange(start, stop, step) → Random number with step.

random.choice(list) → Picks one random element.

random.shuffle(list) → Rearranges list elements randomly.

**(ii) Program to Print Current Date & Time and Use math Functions**

This program uses:

The datetime module to get the current date and time.

The math module for mathematical operations.

```
# Program: Display current date & time and demonstrate math functions

# Import required modules
import datetime
import math

# 1 Display current date and time
current = datetime.datetime.now()
print("Current Date and Time:", current)

# 2 Display only date
print("Today's Date:", current.date())

# 3 Display only time
print("Current Time:", current.time())

# 4 Demonstrate math functions
num = 25.75
print("\nUsing math functions:")
print("Square root of 25:", math.sqrt(25))
print("Ceiling of", num, ":", math.ceil(num))
print("Floor of", num, ":", math.floor(num))
print("Power of 2^5:", math.pow(2, 5))
print("Value of Pi:", math.pi)
print("Value of e (Euler's number):", math.e)
```

**Output:**

```
Current Date and Time: 2026-02-19 11:24:35.726315
Today's Date: 2026-02-19
Current Time: 11:24:35.726315

Using math functions:
Square root of 25: 5.0
Ceiling of 25.75 : 26
Floor of 25.75 : 25
```

Power of 2^5: 32.0
Value of Pi: 3.141592653589793
Value of e (Euler's number): 2.718281828459045

**Explanation:**

datetime.now() → Gets current date & time.

math.sqrt() → Square root.

math.ceil() / math.floor() → Round up or down.

math.pow(a, b) → raised to the power b.

math.pi, math.e → Constants for π and e.

**4) Write a program to create a Student class with attributes, roll number, name, and marks, and display the student details using objects and methods.**

Program: Student Class with Attributes and Methods

```
# Program: Create a Student class and display student details

# Define a class named 'Student'
class Student:
    # Constructor method to initialize attributes
    def __init__(self, roll_no, name, marks):
        # 'self' refers to the current object
        self.roll_no = roll_no
        self.name = name
        self.marks = marks

    # Method to display student details
    def display_details(self):
        print("---- Student Details ----")
        print("Roll Number:", self.roll_no)
        print("Name:", self.name)
        print("Marks:", self.marks)

# Create objects of the Student class
student1 = Student(101, "Urvisha Parmar", 89)
student2 = Student(102, "Aayush Mena", 92)
```

```
# Display details using the method
student1.display_details()
print()  # Blank line for better readability
student2.display_details()
```

Output:
Student Details
Roll Number: 101
Name: Urvisha Parmar
Marks: 89

Student Details
Roll Number: 102
Name: Aayush Mena
Marks: 92

**Explanation:**

Class Definition:

class Student: creates a blueprint for student objects.

Constructor (__init__):

Automatically called when an object is created.

Initializes attributes: roll_no, name, and marks.

Method (display_details()):

Prints details of the student object.

Object Creation:

student1 and student2 are two instances (objects) of the Student class.

Method Call:

Each object calls its display_details() method to show its own data.

**5) Develop a program to demonstrate encapsulation by creating a BankAccount class with private data member balance and methods to deposit, withdraw, and display balance.**

Program: Demonstrate Encapsulation in Python

```
# Program: Demonstrate Encapsulation using a BankAccount class

class BankAccount:
    # Constructor to initialize account with a starting balance
    def __init__(self, initial_balance=0):
        # Private data member (encapsulated)
        self.__balance = initial_balance

    # Method to deposit money
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited: ₹{amount}")
        else:
            print("Invalid deposit amount!")

    # Method to withdraw money
    def withdraw(self, amount):
        if amount > 0 and amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrawn: ₹{amount}")
        elif amount > self.__balance:
            print("Insufficient balance!")
        else:
            print("Invalid withdrawal amount!")

    # Method to display the current balance
    def display_balance(self):
        print(f"Current Balance: ₹{self.__balance}")

# Create a BankAccount object
account1 = BankAccount(1000)   # Initial balance ₹1000

# Display initial balance
account1.display_balance()

# Deposit some money
account1.deposit(500)
```

```
# Try withdrawing money
account1.withdraw(300)

# Try withdrawing more than available
account1.withdraw(1500)

# Display final balance
account1.display_balance()

# Trying to access private variable directly (will not work)
# print(account1.__balance)  #  This will raise an AttributeError
```

**Output:**

Current Balance: ₹1000
Deposited: ₹500
Withdrawn: ₹300
Insufficient balance!
Current Balance: ₹1200

**Explanation:**

Encapsulation:

The variable __balance is declared private (using double underscores __), so it cannot be accessed directly from outside the class.

Access is only allowed through public methods (deposit, withdraw, display_balance).

Data Protection:

This prevents accidental modification of the balance value.

Public Methods:deposit()  Adds money to the account (if valid).

withdraw()  Deducts money only if enough balance.

display_balance()  Safely shows the balance.

Trying to Access Private Variable Directly:

account1.__balance → Raises error (cannot access private data directly).

**6) Write a program to implement inheritance by creating a base class Employee and a derived class Manager, and demonstrate method overriding.**

```
# Program: Implement Inheritance and Method Overriding

# Base class
class Employee:
    def __init__(self, emp_id, name, salary):
        self.emp_id = emp_id
        self.name = name
        self.salary = salary

    # Method to display employee details
    def display_info(self):
        print("---- Employee Details ----")
        print("Employee ID:", self.emp_id)
        print("Name:", self.name)
        print("Salary:", self.salary)

# Derived class (inherits from Employee)
class Manager(Employee):
    def __init__(self, emp_id, name, salary, department):
        # Call the constructor of the base class using super()
        super().__init__(emp_id, name, salary)
        self.department = department

    # Method overriding — redefining the base class method
    def display_info(self):
        print("---- Manager Details ----")
        print("Employee ID:", self.emp_id)
        print("Name:", self.name)
        print("Salary:", self.salary)
        print("Department:", self.department)

# Create an Employee object
emp1 = Employee(101, "Urvisha Parmar", 35000)

# Create a Manager object (inherits from Employee)
mgr1 = Manager(201, "Aayush Mena", 55000, "IT Department")

# Display employee details
emp1.display_info()
print()  # Blank line for clarity
```

```
# Display manager details (method overriding in action)
mgr1.display_info()
```

**Output:**

```
-Employee Details
Employee ID: 101
Name: Urvisha Parmar
Salary: 35000

Manager Details
Employee ID: 201
Name: Aayush Mena
Salary: 55000
Department: IT Department
```

Explanation:

Inheritance Concept:

Manager inherits properties and methods from Employee.

super() is used to call the base class constructor inside the derived class.

Method Overriding:

Both classes have a method named display_info().

In Manager, this method is redefined to include department details.

When called on a Manager object, the derived version overrides the base version.

Output Difference:

The Employee object prints basic details.

The Manager object prints extra details (department) because of overriding.

**7) (i) Program to Perform Basic File Handling Operations in Python**

File handling allows reading from and writing to files.
Here, we'll perform operations: create, write, read, and append.

```
# Program: Basic File Handling Operations in Python

# 1 Create and write to a new file
file = open("student.txt", "w")   # 'w' mode = write (creates file if not exists)
file.write("Name: Urvisha Parmar\n")
file.write("Course: BCA\n")
file.write("Marks: 89\n")
file.close()
print("File created and data written successfully.\n")

# 2 Read the content of the file
file = open("student.txt", "r")   # 'r' mode = read
print("---- File Content ----")
print(file.read())              # Read entire file content
file.close()

# 3 Append new data to the file
file = open("student.txt", "a")   # 'a' mode = append
file.write("Result: Pass\n")
file.close()
print("Data appended successfully.\n")

# 4 Read again to show updated content
file = open("student.txt", "r")
print("---- Updated File Content ----")
print(file.read())
file.close()
```

Output:
File created and data written successfully.

File Content
Name: Urvisha Parmar
Course: BCA
Marks: 89

Data appended successfully.

- Updated File Content -
Name: Urvisha Parmar
Course: BCA
Marks: 89
Result: Pass

Explanation:

open(filename, mode) → Opens a file in specified mode:

'w' → write

'r' → read

'a' → append

file.write() → Writes text into the file.

file.read() → Reads file content.

file.close() → Closes the file to free memory.

**(ii) Program: Opening a File That Is Not Writable (Using Exception Handling)**

This program demonstrates how to handle errors when trying to write to a read-only file using try–except blocks.

```
# Program: Exception Handling for Non-writable File

try:
    # Open file in read mode only (not writable)
    file = open("readonly.txt", "r")

    # Try writing to it — this will raise an exception
    file.write("This should cause an error!")
    file.close()

except IOError:
    print("Error: File is not writable! (IOError occurred)")

finally:
    print("Program executed with exception handling.")
```

**Output:**
Error: File is not writable! (IOError occurred)
Program executed with exception handling.

Explanation:

try: → Code that might cause an error.

except IOError: → Catches input/output related errors.

finally: → Runs whether an error occurs or not (cleanup or message).

In this example, writing to a file opened in 'r' mode causes an IOError, which is safely handled by except.

***steps tkinter : STEPS TO RUN TKINTER PROGRAM IN PYCHARM***
### Step 1: Install Python
* Download from python.org
* Install and ✔ Tick **Add Python to PATH**

### Step 2: Install PyCharm

* Install PyCharm Community Edition
* Open PyCharm

### Step 3: Create New Project

* Click **New Project**
* Select location
* Click **Create**

### Step 4: Create Python File

* Right click Project → **New → Python File**
* Name it (example: program8.py)

### Step 5: Paste Code and Run

* Right click file → Run
  OR
* Click ▶ Run button

**Practical 8: GUI Programming using Tkinter – Draw a Star**

```
# Step 1: Import Tkinter library for GUI
from tkinter import *

# Step 2: Create the main window
root = Tk()

# Step 3: Set title of window
root.title("Draw Star using Tkinter")

# Step 4: Set window size
root.geometry("400x400")

# Step 5: Create canvas where shapes can be drawn
canvas = Canvas(root, width=400, height=400, bg="white")
canvas.pack()

# Step 6: Define coordinates for star shape
points = [200,50, 240,150, 350,150, 260,220,
        300,330, 200,260, 100,330, 140,220,
        50,150, 160,150]

# Step 7: Draw star using polygon
canvas.create_polygon(points, fill="yellow", outline="black", width=2)

# Step 8: Run the GUI window
root.mainloop()
```

**Output**

A **GUI window appears (400x400)** displaying a **yellow star drawn in the center on a white background**.

**Practical 9: Registration Form using Tkinter**

```
# Import Tkinter
from tkinter import *

# Function to display entered data
def submit():
   print("Name:", name.get())
   print("Email:", email.get())
   print("Gender:", gender.get())
   print("Course:", course.get())

# Create window
root = Tk()
root.title("Student Registration Form")
root.geometry("350x300")

# Heading
Label(root,text="Registration Form",font=("Arial",14)).pack()

# Variables for storing data
name = StringVar()
email = StringVar()
gender = StringVar()
course = StringVar()

# Name field
Label(root,text="Name").pack()
Entry(root,textvariable=name).pack()

# Email field
Label(root,text="Email").pack()
Entry(root,textvariable=email).pack()

# Gender selection
Label(root,text="Gender").pack()
Radiobutton(root,text="Male",variable=gender,value="Male").pack()
Radiobutton(root,text="Female",variable=gender,value="Female").pack()

# Course field
Label(root,text="Course").pack()
Entry(root,textvariable=course).pack()
```

```
# Submit button
Button(root,text="Submit",command=submit).pack()

root.mainloop()
```

**Output**

GUI Form appears.
Example console output after clicking submit:
Name: Rahul
Email: rahul@gmail.com
Gender: Male
Course: BCA

**Practical 10: Check Box and Radio Button**

```
from tkinter import *

# Function to display selection
def show():
  print("Selected Mode:", mode.get())

  if python_var.get() == 1:
     print("Python Selected")

  if java_var.get() == 1:
     print("Java Selected")

# Create window
root = Tk()
root.title("Checkbox and Radiobutton Example")

# Variable for radio button
mode = StringVar()

Label(root,text="Select Mode").pack()

# Radio buttons (only one selection allowed)
Radiobutton(root,text="Online",variable=mode,value="Online").pack()
Radiobutton(root,text="Offline",variable=mode,value="Offline").pack()

# Variables for checkbox
python_var = IntVar()
java_var = IntVar()
```

```
Label(root,text="Select Subject").pack()

# Checkboxes (multiple selections allowed)
Checkbutton(root,text="Python",variable=python_var).pack()
Checkbutton(root,text="Java",variable=java_var).pack()

Button(root,text="Submit",command=show).pack()

root.mainloop()
```

**Output**

Example console output:
Selected Mode: Online
Python Selected
Java Selected

**Practical 11 (i): Listbox with Scrollbar**

```
from tkinter import *

# Create window
root = Tk()
root.title("Listbox with Scrollbar")

# Create scrollbar
scrollbar = Scrollbar(root)
scrollbar.pack(side=RIGHT, fill=Y)

# Create listbox
listbox = Listbox(root, yscrollcommand=scrollbar.set)

# Insert items
languages = ["Python","Java","C","C++","HTML","CSS",
             "JavaScript","PHP","AI","Machine Learning"]

for item in languages:
  listbox.insert(END,item)

listbox.pack()

# Configure scrollbar
scrollbar.config(command=listbox.yview)

root.mainloop()
```

**Output**
GUI window shows:
Python
Java
C
C++
HTML
CSS
JavaScript
PHP
AI
Machine Learning
with a scrollbar **on the right side**.

**Practical 11 (ii): Message Box**

```
from tkinter import *
from tkinter import messagebox

# Function to show message boxes
def show_message():

  # Information message
  messagebox.showinfo("Information","Welcome to Python GUI")

  # Warning message
  messagebox.showwarning("Warning","Check your input")

  # Error message
  messagebox.showerror("Error","Something went wrong")

# Create window
root = Tk()
root.title("Message Box Example")

Button(root,text="Show Message",command=show_message).pack()

root.mainloop()
```

**Output**
Three popup messages appear sequentially:

1 Information → *Welcome to Python GUI*
2 Warning → *Check your input*
3 Error → *Something went wrong*

**Practical 12: List and Dictionary Comprehension**

```
# Create list of numbers
numbers = [1,2,3,4,5]

# List comprehension (square values)
squares = [x**2 for x in numbers]

print("Squares List:",squares)

# Dictionary comprehension
square_dict = {x:x**2 for x in numbers}

print("Squares Dictionary:",square_dict)
```

**Output**
Squares List: [1, 4, 9, 16, 25]
Squares Dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

**Practical 13: Stack using Python List**

```
# Create empty stack
stack = []

# Push operation
def push():
  element = input("Enter element to push: ")
  stack.append(element)
  print("Stack:",stack)

# Pop operation
def pop():
  if len(stack) == 0:
     print("Stack is empty")
  else:
     print("Popped element:",stack.pop())

# Peek operation
```

```
def peek():
  if len(stack) == 0:
      print("Stack empty")
  else:
      print("Top element:",stack[-1])

# Check if empty
def isEmpty():
  if len(stack) == 0:
      print("Stack is empty")
  else:
      print("Stack is not empty")

push()
push()
peek()
pop()
isEmpty()
```

**Output Example**

```
Enter element to push: 10
Stack: ['10']

Enter element to push: 20
Stack: ['10', '20']

Top element: 20
Popped element: 20
Stack is not empty
```

**Practical 14: Queue using Python List**

```
# Create empty queue
queue = []

# Enqueue operation
def enqueue():
  element = input("Enter element: ")
  queue.append(element)
  print("Queue:",queue)

# Dequeue operation
def dequeue():
  if len(queue) == 0:
```

```
        print("Queue is empty")
    else:
        print("Removed:",queue.pop(0))

# Peek operation
def peek():
    if len(queue) == 0:
        print("Queue empty")
    else:
        print("Front element:",queue[0])

# Queue size
def size():
    print("Queue size:",len(queue))

enqueue()
enqueue()
peek()
dequeue()
size()
```

**Output**

```
Enter element: 5
Queue: ['5']

Enter element: 10
Queue: ['5', '10']

Front element: 5
Removed: 5
Queue size: 1
```